

Cryptography 1 Lab - Basic Cryptography and Cryptanalysis

9 September 2026

This lab is designed to help you learn and practice the fundamental concepts of cryptography, including classical ciphers, brute-force attacks, frequency analysis, cryptographic hashing, XOR encryption, One-Time Pads, and Diffie–Hellman key exchange.

Learning Objectives

By the end of this exercise session, students will be able to:

- Explain the basic concepts of encryption, keys, and key spaces.
- Implement and use a Caesar cipher.
- Perform a brute-force attack against a weak cipher.
- Implement a monoalphabetic substitution cipher and a matrix transposition cipher.
- Understand cryptographic hashing and calculate and compare SHA-256 hashes
- Understand XOR encryption and One-Time Pad.
- Implement a basic Diffie–Hellman key exchange in Java.

Exercise 1: Caesar Cipher

Implement Caesar cipher encryption and decryption functions in Python. Your program should get a message and an integer key from the user. It should encrypt the message and decrypt the ciphertext using the same key.

Verify that decrypting an encrypted message produces the original message by testing it with several different messages and keys.

Encrypt the following message using a Caesar cipher with a key of 7:

a) *If you can meet with triumph and disaster, and treat those two impostors just the same.*

Decrypt the following message using the Caesar cipher with a key of 15:

b) *Lt wd as iwtht igjiwh id qt htau-tkxstci, iwpi paa btc pgt rgtpits tfjpa, iwpi iwtn pgt tcsdlts qn iwtxg Rgtpidg lxiw rtgipxc jcpaxtcpqat Gxvwih, iwpi pbdcv iwtht pgt Axut, Axqtgin pcs iwt ejghjxi du Wpeexcthh.*

Extra: Decrypt the following ciphertexts using a Caesar cipher. Determine the key for each ciphertext.

- c) LWKLQNWKDWLVKDOOQHYHUVHHDELOOERDUGORYHOBDDVD
WUHH
- d) UXENRBWXCUXENFQRLQJUCNABFQNWRCJUCNAJCRXWORWM
B
- e) BGUTBMBGZTFHNLXMKTIPBMAVAXXLXTEPTRLEXTOKHHFYHK
MAXFHNLX
- f) LUXDZNUAMNDODJUDTUZDGYQDLUXDGOJDCKDTKKJDOZ

1.1 Brute-Force Attack on a Caesar cipher

A Caesar cipher has a very small number of possible keys, e.g., for the English alphabet, there are only 26 possible shifts.

Instead of knowing the key, assume that you only have the ciphertext.

Write a Python program that gets a ciphertext from the user and tries every possible Caesar cipher key and displays the resulting plaintext for each key. Test it using some ciphertext (e.g., KHOOR ZRUOG)

Exercise 2: Transposition Cipher

Implement a matrix transposition cipher in Python.

Use a matrix with a user-specified number of columns.

For encryption, remove spaces, write the plaintext row-by-row into the matrix, and pad the final row if necessary (e.g., x). Read the matrix column-by-column to produce the ciphertext.

For decryption, use the same number of columns as the encryption process. Determine the number of rows from the ciphertext length, place the ciphertext into the matrix column-by-column, then read the matrix row-by-row. Remove the padding and verify that the original plaintext is recovered.

Exercise 3: Substitution Cipher

Create your own monoalphabetic substitution cipher in Python. Your program should either generate or accept a substitution alphabet and encrypt a plaintext message. Decrypt a ciphertext using the substitution key. Test it by encrypting a paragraph of at least 100 characters.

EXTRA: Exchange your ciphertext with another group. Attempt to recover the plaintext without being given the substitution key.

Use frequency analysis, common English words, repeated-letter patterns, and common digrams/trigrams.

Exercise 4. XOR Encryption

Implement an XOR encryption/decryption function in Python. Your program should accept a plaintext message and an XOR key to encrypt the plaintext. It should decrypt the ciphertext using the same key and verify that the original plaintext is recovered.

Experiment with a one-byte key, a short repeating key, a key longer than the plaintext and different plaintext messages.

Exercise 5. One-Time Pad

Write a Python function that generates a cryptographically secure random key with the same length as the plaintext.

Use

```
import secrets
```

(The key must be generated randomly, must be at least as long as the plaintext, and must never be reused.)

Encrypt a message, e.g., "This is a simple secret message", and decrypt the ciphertext using the same OTP key and verify that the original plaintext is recovered.

Exercise 5. Cryptographic Hashing

In this exercise, use **SHA-256** with Python's hashlib library.

Write a Python program that calculates a SHA-256 hash.

Calculate the SHA-256 hash of each of the following strings:

password

Password

password1

password2

Display both the input and its corresponding hash.

Determine the output size of SHA-256 in:

- bits
- bytes
- hexadecimal characters

Exercise 6. Implement Diffie Hellman key exchange in Java

Implement a basic Diffie–Hellman key exchange in Java using Java's cryptographic APIs.

Use *KeyPairGenerator* with the algorithm *DH* and a parameter size of 2048 bits.

Generate a DH key pair for Alice and a separate key pair for Bob. Exchange their public keys. Use *KeyAgreement* to derive the shared secret for Alice.

Use *KeyAgreement* to derive the shared secret for Bob. Verify that both shared secrets are identical.